
Lab Assignment # 1

Name of Student : CHATHARABOINA SAGAR
Enrollment No. : 2303A51522
Batch No. : 22

Task 1: AI-Generated Logic Without Modularization (Factorial without Functions)

Prompt:-

```
Write a Python Code to calculate the factorial of a number
# entered by the user using loops only. Do not use any functions
# The entire logic should be written in one place, making it easy for
beginners to understand.
```

Code:-

```
lab2.py > ...
1 #-----Task_1/5-----
2 #Write a Python Code to calculate the factorial of a number
3 # entered by the user using loops only. Do not use any functions
4 # The entire logic should be written in one place, making it easy for beginners to understand.
5
6 n=int(input("Enter a number: "))
7 if n < 0:
8     print("Factorial not defined for negative numbers")
9 elif n == 0 or n == 1:
10    print("The factorial of", n, "is 1")
11 else:
12     result = 1
13     for i in range(2, n + 1):
14         result *= i
15    print("The factorial of", n, "is", result)
```

Output:-

```
PS C:\sagar\pack1\OneDrive\3-2\AI Assistant Coding> & C:/Users/dell/AppData/Local/Programs/Python/Python312/python.exe "c:
/sagar/pack1/OneDrive/3-2/AI Assistant Coding/lab2.py"
Enter a number: 5
The factorial of 5 is 120
```

Justification:-

This program solves the problem in a clear and straightforward way by writing all the logic in one place without using functions. It is easy for beginners to understand and suitable for small tasks. However, since functions are not used, the code is not reusable or flexible for larger programs.

Task 2: AI Code Optimization & Cleanup (Improving Efficiency)

Prompt:-

```
#-----Task_2/5-----
```

```
#Write a Python Code to calculate the factorial  
of a number  
#Simplify and clean the factorial program to make it more readable and  
efficient by removing unnecessary parts.
```

Code:-

```
#-----Task_2/5-----  
#Write a Python Code to calculate the factorial of a number  
#Simplify and clean the factorial program to make it more readable and  
# efficient by removing unnecessary parts.  
def factorial(n):  
    if n < 0:  
        return "Factorial not defined for negative numbers"  
    elif n == 0 or n == 1:  
        return 1  
    else:  
        result = 1  
        for i in range(2, n + 1):  
            result *= i  
        return result
```

Output:-

```
PS C:\sagar\pack1\OneDrive\3-2\AI Assistant Coding> & C:/Users/dell/AppData/Local/Programs/Python/Python312/python.exe "c:  
:\sagar\pack1\OneDrive\3-2\AI Assistant Coding\lab2.py"  
Enter a number: 7  
The factorial of 7 is 5040
```

Justification:-

This improved version of the code is cleaner and easier to read. By removing unnecessary parts and keeping only essential logic, the program becomes easier to understand, debug, and maintain, especially when working on larger projects or with a team.

(Factorial with Functions)

Prompt:-

```
#-----Task 3/5-----  
#Write a Python program to calculate the factorial of a number using a  
function.  
# The factorial logic should be placed inside a separate function to make the  
program more  
# structured, reusable, and professional. This approach should clearly  
separate the main program flow from the logic,  
# making it easier to modify and suitable for larger programs.
```

Code:-

```
#-----Task 3/5-----  
#Write a Python program to calculate the factorial of a number using a function.  
# The factorial logic should be placed inside a separate function to make the program more  
# structured, reusable, and professional. This approach should clearly separate the main program flow from the logic,  
# making it easier to modify and suitable for larger programs.  
num = int(input("Enter a number: "))  
fact = factorial(num)  
print(fact)  
def factorial(n):  
    if n < 0:  
        return "Factorial not defined for negative numbers"  
    elif n == 0 or n == 1:  
        return 1  
    else:  
        result = 1  
        for i in range(2, n + 1):  
            result *= i  
        return result
```

Output:-

```
PS C:\sagar\pack1\OneDrive\3-2\AI Assistant Coding> & C:/Users/dell/AppData/Local/Programs/Python/Python312/python.exe "c:/s  
sagar/pack1/OneDrive/3-2/AI Assistant Coding/lab2.py"  
Enter a number: 6  
The factorial of 6 is 720
```

Justification:-

Using a function makes the program more organized and professional. It separates the factorial logic from the main code, allowing the logic to be reused

Task 4: Comparative Analysis – Procedural vs Modular AI Code (With vs Without Functions)

Prompt:-

```
#-----Task 4/5-----
#Compare factorial programs written with and without functions.
# Explain which approach is better for understanding, reusability, debugging,
and use in large projects,
# and present the comparison in a small table
```

Code:-

```
lab2.py > factorial_with_function
51 #-----Task 4/5-----
52 #Compare factorial programs written with and without functions.
53 # Explain which approach is better for understanding, reusability, debugging, and use in large
54 # and present the comparison in a small table
55 import math
56 def factorial_with_function(n):
57     if n < 0:
58         return "Factorial not defined for negative numbers"
59     elif n == 0 or n == 1:
60         return 1
61     else:
62         result = 1
63         for i in range(2, n + 1):
64             result *= i
65         return result
```

Output:-

```
"S C:\sagar\pack1\OneDrive\3-2\AI Assistant Coding> & C:/Users/dell/AppData/Local/Programs/Python/Python312/python.exe "c:/s
sagar\pack1\OneDrive\3-2\AI Assistant Coding/lab2.py"
Enter a number: -1
Factorial not defined for negative numbers
Enter a number: 4
24
```

Justification:-

Comparing both approaches helps understand the advantages of using functions. It shows why modular code is easier to understand, reuse, debug, and manage in large projects compared to procedural code written without functions.

Task 5: AI-Generated Iterative vs Recursive Thinking

Prompt:-

```
#Write a Python program to calculate factorial using both iterative and recursive methods.  
# Explain how each method works and compare them based on readability, memory usage, performance, and limitations.
```

Code:-

```
#----Task 5/5-----  
#Write a Python program to calculate factorial using both iterative and recursive methods.  
# Explain how each method works and compare them based on readability, memory usage, performance, and limitations.  
def factorial_iterative(n):  
    if n < 0:  
        return "Factorial not defined for negative numbers"  
    elif n == 0 or n == 1:  
        return 1  
    else:  
        result = 1  
        for i in range(2, n + 1):  
            result *= i  
        return result  
def factorial_recursive(n):  
    if n < 0:  
        return "Factorial not defined for negative numbers"  
    elif n == 0 or n == 1:  
        return 1  
    else:  
        return n * factorial_recursive(n - 1)  
num = int(input("Enter a number: "))  
print(f"Factorial of {num} using iterative method: {factorial_iterative(num)}")  
print(f"Factorial of {num} using recursive method: {factorial_recursive(num)}")  
print(fact)
```

Output:-

```
Enter a number: 7  
Factorial of 7 using iterative method: 5040  
Factorial of 7 using recursive method: 5040  
120  
PS C:\sagar\pack1\OneDrive\3-2\AI Assistant Coding> 
```

Justification:-

This task helps understand the difference between iterative and recursive approaches. By comparing both methods, it becomes easier to see their strengths, weaknesses, and when each method should be used in real programs.

